

7.6.3 处理器亲和性

在多核调度中，操作系统能够根据特定策略在 CPU 核心间迁移任务，从而达到负载均衡的目的。然而，有时候开发者并不希望调度器这么做。例如，小明现在已经是一位有经验的开发者，他完全熟悉自己的程序和调度器的设计。他希望自己程序的任务只在他指定的 CPU 核心上执行。

为了使开发者能够控制自己程序的执行，Linux 和 ChCore 等操作系统都提供了处理器亲和性（Processor Affinity）的机制，允许程序对任务可以使用的 CPU 核心进行配置。任务可设置 `cpu_set_t` 掩码，用于表示可以执行任务（以线程为单位）的 CPU 核心集合，掩码中的每一位都对应一个 CPU 核心。如代码片段 7.4 所示，提供了一系列对 `cpu_set_t` 进行操作的代码宏。

代码片段 7.4 操作 CPU 核心集合的代码宏

```

1 #include <sched.h>
2
3 // 对 set 进行初始化，设置为空
4 void CPU_ZERO(cpu_set_t *set);
5 // 将对应 CPU 核心加入 set 中
6 void CPU_SET(int cpu, cpu_set_t *set);
7 // 将对应 CPU 核心从 set 中删除
8 void CPU_CLR(int cpu, cpu_set_t *set);
9 // 判断对应 CPU 核心是否在 set 中，是则返回 1，否则返回 0
10 int CPU_ISSET(int cpu, cpu_set_t *set);
11 // 返回当前 set 中的 CPU 核心数量
12 int CPU_COUNT(cpu_set_t *set);
13 ...

```

在这些操作宏的基础上，操作系统提供了设置和获取任务亲和性的系统调用，如代码片段 7.5 所示。程序可以通过 `sched_setaffinity` 设置任务对于 CPU 核心的亲和性，或是通过 `sched_getaffinity` 获取任务的亲和性配置。需要注意的是，由于线程是 Linux 的调度单元，这里的 `pid` 参数实际上是 Linux 为每个线程分配的标识符，如果 `pid` 为 0，代表操作对象是当前的调用者线程。

代码片段 7.5 Linux 的任务（线程）亲和性接口

```

1 #include <sched.h>
2
3 int sched_setaffinity(pid_t pid, size_t cpusetsize,
4                      const cpu_set_t *mask);
5 int sched_getaffinity(pid_t pid, size_t cpusetsize,
6                      cpu_set_t *mask);

```

代码片段 7.6 展示了设置任务亲和性的示例代码。设置完成后，该任务只能在 CPU 核心 0 和 2 上执行。操作系统在进行任务调度和迁移时，会检查其迁移目标是否在该任

务允许使用的 CPU 核心集合（包含 0 和 2 两个 CPU 核心）中，如果不在，则不会迁移任务。

代码片段 7.6 设置线程亲和性的代码示例

```
1 #include <sched.h>
2 #include <stdio.h>
3 #include <sys/sysinfo.h>
4
5 int main(int argc, char *argv)
6 {
7     cpu_set_t mask;
8     // 初始化 mask 的 CPU 集合为空
9     CPU_ZERO(&mask);
10
11     // 在 mask 的 CPU 集合中加入 CPU 核心 0 和 CPU 核心 2
12     CPU_SET(0, &mask);
13     CPU_SET(2, &mask);
14
15     // 根据 mask 设置当前线程的亲和性
16     sched_setaffinity(0, sizeof(mask), &mask);
17
18     // 执行程序逻辑
19     ...
20
21     return 0;
22 }
```

7.7 案例分析：Linux 调度器

本节通过案例分析的方式介绍 Linux 发展历史上使用过或正在使用的调度器与相关机制。Linux 诞生伊始，开发重点并非处理器调度。当时对调度器的要求是：实现简单，确保任务不会饥饿。因此，Linux 在 1.2 版本时使用了时间片轮转策略，将一个全局的环形队列作为调度器的运行队列。后来，随着计算机的处理能力变强，以及对称多处理架构（Symmetric Multi-Processor, SMP）的出现，用户将越来越多的任务交由 Linux 处理，处理器调度面对的工作场景越来越复杂。时间片轮转策略仅仅考虑了任务的执行时间，无法满足复杂场景所带来的日益严苛的调度需求。

具体来说，Linux 调度器需要考虑非常多的因素（以及它们对应的需求），包括但不限于：

- 任务的种类。Linux 将任务分为实时任务、交互式任务和批处理任务，并且调度器必须保证实时任务优先于其他两类任务执行。
- 任务使用的资源。根据不同任务使用的主要资源的不同，可以粗略地将任务分类为计算密集型任务和 I/O 密集型任务。为了尽可能保证计算机整体的资源利用率，

调度器应该优先调度 I/O 密集型任务。

- 任务的等待时间。由于各种因素的存在，某些任务可能等待了很长时间而没有被执行。为了调度的公平性，调度器应该调度那些等待时间过长的任务。

此外，还有大量其他因素，包括本章未涉及的多核调度相关的因素。那么，该如何设计一个涵盖这么多因素的调度器呢？

处理器调度的目的就是决定哪个任务更应该被执行。换句话说，处理器调度就是要确定哪个任务优先级高，并调度它。Linux 统一用任务的优先级来衡量上述这些因素，尽管因素非常复杂，但对于调度器来说，就是在比较任务的优先级。根据上述思想，从 1.2 版本开始，Linux 调度器不断演化，Linux 2.4 版本开始使用 $O(N)$ 调度器。

7.7.1 $O(N)$ 调度器

顾名思义， $O(N)$ 调度器指定调度决策的时间复杂度是 $O(N)$ ， N 代表调度器运行队列中的任务数量。我们将首先介绍 $O(N)$ 调度器的设计，然后讨论这一设计所带来的问题。

$O(N)$ 调度器的思想是找出所有任务中优先级最高的任务。由于任务因素种类繁多，其中不乏动态变化的因素，例如任务主要使用的资源和等待时间，因此，Linux 的调度器（包括后文介绍的两个 Linux 调度器）会在任务运行时计算任务的动态优先级。对于需要立即执行的实时任务，Linux 会保证实时任务的动态优先级高于非实时任务（交互式任务和批处理任务）的动态优先级。而在计算非实时任务间的动态优先级时，本节讨论的 $O(N)$ 调度器会综合考虑任务的各种因素等。调度器的“ $O(N)$ ”体现在：当开始调度决策时，需要遍历运行队列中的所有任务，并且重新计算它们的动态优先级，然后选取动态优先级最高的任务执行。

虽然任务对应动态优先级，但是为了公平性，Linux 为非实时任务设置了时间片，保证不会产生任务饥饿。早期的计算机硬件资源有限，任务上下文切换的开销非常大，因此 Linux 倾向于为任务设置尽可能长的时间片。但是，过长的时间片又与任务的低响应时间需求产生了冲突。为了尽可能权衡时间片长短带来的利弊，Linux 的调度器会将时间分为多个调度时间段（Epoch）。每经过一个时间段，调度器都会重新分配任务的时间片，避免所有任务全部执行一次的总时间片过长，导致任务的响应时间过长。同时，为了弥补那些在一个调度时间段结束后仍有剩余时间片的任务，这些任务的剩余时间片的一半会被加入它们的下一个时间片中。

在 Linux 早期，一个系统内的任务数量很少， $O(N)$ 复杂度仍然能够保证系统正常工作。但是随着操作系统和硬件的不断发展，一个系统内需要管理的任务越来越多， $O(N)$ 调度器设计中的问题也逐渐暴露出来：调度开销过大。 $O(N)$ 时间复杂度会导致在任务过多的情况下调度器的决策时间过长，浪费 CPU 资源。同时，在所有任务执行完一个时间片后， $O(N)$ 调度器需要更新它们的时间片，这也会造成额外的调度开销。为

此, Linux 在 2.6.0 版本中使用新的 $O(1)$ 调度器替换了 $O(N)$ 调度器。

7.7.2 $O(1)$ 调度器

相比于 $O(N)$ 调度器, $O(1)$ 调度器的“ $O(1)$ ”体现在: Linux 限制了任务的优先级最多只能有 140 个, 其中实时任务对应高优先级范围 $[0, 100)$, 非实时任务对应低优先级范围 $[100, 140)$, 在给定任务类型的优先级范围中, 用户可具体指定任务的优先级。同时, $O(1)$ 调度器的运行队列可以被视为多级队列, 140 个优先级队列分别对应一个优先级。多级队列还维护了一个位图 (Bitmap), 位图中的比特用于判断对应的优先级队列是否有任务等待调度。在制定调度决策时, $O(1)$ 调度器会根据位图找到队列中第一个不为空的队列, 并调度该队列的第一个任务, 其时间复杂度为 $O(1)$, 与运行队列中的任务数量无关。

具体来说, 运行队列实际上是由两个多级队列组成的——激活队列 (Active Queue) 和过期队列 (Expired Queue), 分别用于管理仍有时间片剩余的任务和时间片耗尽的任务。当一个任务的时间片耗尽后, 它会被加入过期队列中。如果当前激活队列中没有可调度的任务, $O(1)$ 调度器会将两个队列的角色互换, 开始新一轮调度。另外, 用户可能不希望交互式任务在一次时间片用完后, 就要等待所有其他任务的时间片用完才能再次执行。因此该类任务在时间片用完后, 仍然会被加入激活队列中。同时, 为了防止交互式任务过于激进地让当前过期队列中的任务无法执行, 当过期队列的任务等待了过长时间后, $O(1)$ 调度器会把交互式任务加入过期队列中。

尽管 $O(1)$ 调度器的调度开销很小, 而且在大部分场景下都能获得不错的性能, 但仍然存在以下弊端。

首先是交互式任务的判定算法过于复杂。 $O(1)$ 调度器为了保证交互式任务被优先调度, 会将执行完一个时间片的交互式任务重新放回激活队列中。然而复杂系统中的任务可能会在不同任务种类中动态切换, 因此 $O(1)$ 调度器运用了大量启发式方法来判定一个任务是不是交互式任务。在大部分场景下, $O(1)$ 调度器的启发式方法判断交互式任务比较准确, 而在特定场景下则不然, 导致 $O(1)$ 调度器在特定场景下的交互式任务无法及时响应用户。另外, 这些启发式方法过于复杂, 在代码中硬编码了大量参数, 使得代码的可维护性很差。

其次是时间片分配带来的问题。 $O(1)$ 调度器中非实时任务的运行时间片是根据其静态优先级^①确定的, 例如静态优先级最高和最低的非实时任务, 它们的时间片分别为 800 毫秒和 5 毫秒。 $O(1)$ 调度器分配给优先级高的任务的时间片更长, 这就与“批处理任务与交互式任务相比, 优先级更低而需要的时间片更长”的常识相违背。另外, 随

① Linux 同时支持用户指定非实时任务的静态优先级, 对应的概念在 Linux 中称为 Niceness, 即一个任务的友善程度。任务的 Niceness 越高, 对其他任务来说越友善, 则其静态优先级就越低。

着系统中任务数量的增加，任务的调度时延（任务从被放入运行队列到下一次被调度的时延）也会增加，响应时间也会受到影响。例如，两个时间片为 100 毫秒的任务被 $O(1)$ 调度器管理时，它们的调度时延为 100 毫秒；而有三个时间片为 100 毫秒的任务时，调度时延为 200 毫秒。

为解决 $O(1)$ 调度器的问题，Linux 从 2.6.23 版本开始，使用完全公平调度器 (Completely Fair Scheduler, CFS) 替代了 $O(1)$ 调度器。

7.7.3 完全公平调度器

CFS 是目前 Linux 使用的默认调度器。在 7.5 节，我们介绍了公平共享调度策略，该策略保证了每个任务可以根据自己所占的份额共享 CPU 时间，这也是 CFS 的基本思想。之前的 $O(1)$ 调度器需要通过启发式方法确定交互式任务，再给交互式任务更多的执行机会（即将交互式任务重新放回激活队列）。而 CFS 只关心非实时任务对于 CPU 时间的公平共享，避免了复杂的调度算法实现与调参；同时，其通过动态设定任务时间片，确保任务的调度时延不会过高。因此，CFS 无须和 $O(1)$ 调度器一样对交互式任务进行额外的判定和处理。

图 7.18 给出了 CFS 运行队列示意图，Linux 为每个 CPU 核心分配统一的运行队列结构 (rq)，其中的 cfs 指针指向专属于 CFS 的运行队列实现 (cfs_rq)。Linux 中的线程对应调度器中的任务 (task_struct)，其中每个任务的调度实体 (sched_entity) 数据结构维护了调度该任务所需的信息，也是调度器实际操作的对象。

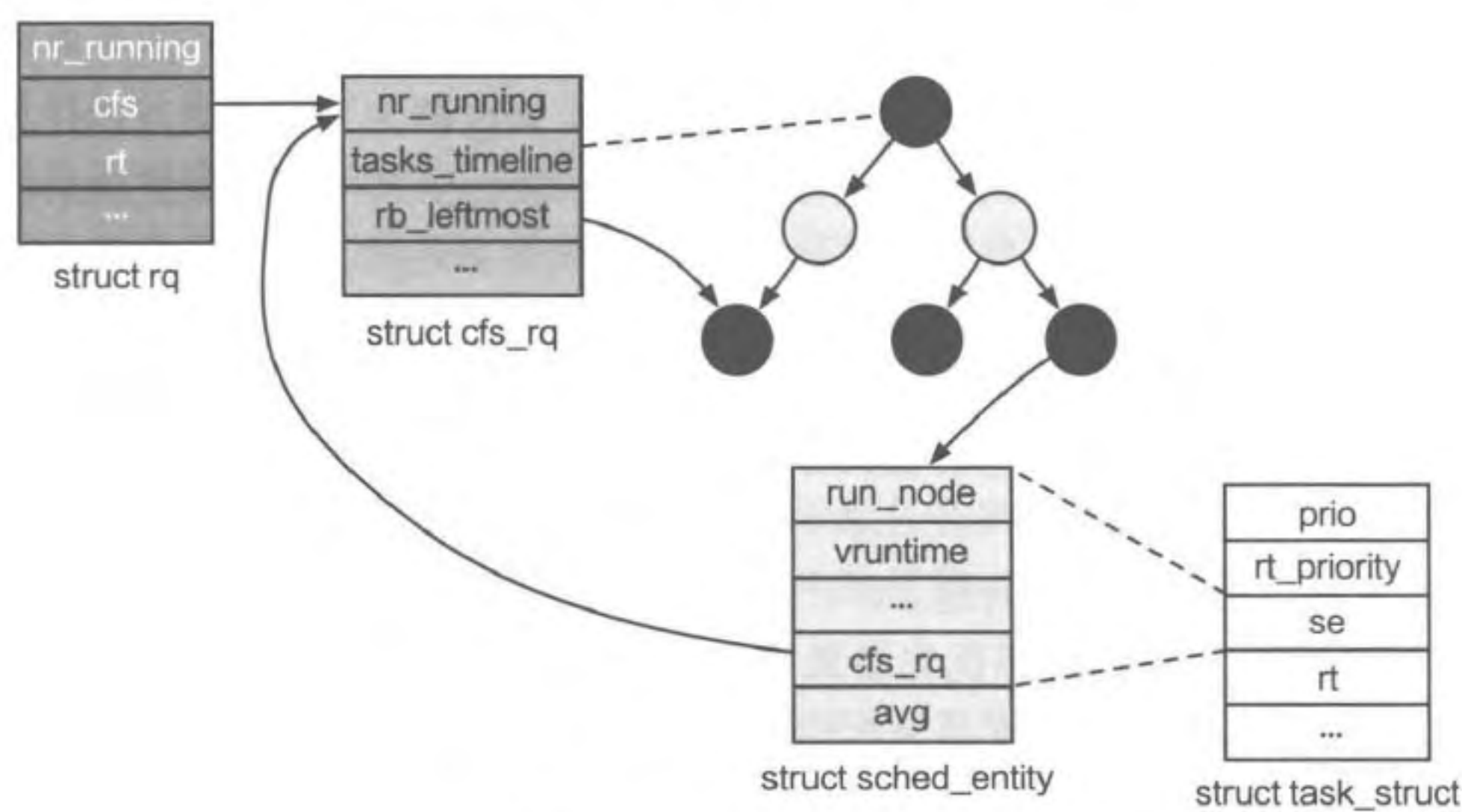


图 7.18 CFS 运行队列示意图

CFS 所使用的调度策略类似于 7.5.2 节介绍的步幅调度。调度实体中维护了该任务

经过的虚拟时间 (vruntime)，与步幅调度的 pass 对应，CFS 会选择虚拟时间最短的任务进行调度。同时，CFS 静态设置非实时任务的静态优先级与任务权重 (Weight，即份额) 的对应关系^①，静态优先级越高则任务的权重越高，可被分配的 CPU 时间越多。

CFS 的动态时间片

为了避免 $O(1)$ 调度器静态设置任务时间片带来的问题 (7.7.2 节)，CFS 使用了调度周期 (sched_period) 的概念，并保证每经过一个调度周期，所有在运行队列中的任务都会被调度一次，从而避免了任务调度时延过长。

具体来说，在 CFS 中，调度周期被设置为运行队列中所有任务各执行一个时间片所需的总时间。因此，每个任务的 CPU 时间占比相当于该任务在一个调度周期内的运行时间在该调度周期中的占比。同时间片一样，调度周期也需要权衡。如果调度周期过长，则一系列任务必须在很长时间的运行后才能体现公平性，且任务的调度时延可能过长；如果调度周期过短，则任务的调度开销会变大。CFS 将调度周期默认设置为 6 毫秒。然而，当系统的任务数量过多时，如果仍使用默认值，那么每个任务分得的时间片较短，可能造成调度开销变大。因此 CFS 还限制了任务平均时间片的最小值，其值为 0.75 毫秒。当默认调度周期不足以满足任务平均最小时间片要求时，调度周期会被设置为当前系统可运行任务数量与 0.75 毫秒的乘积。

此外，在 CFS 调度器确定调度周期后，还需要根据每个任务的权重，对一个调度周期内每个任务的时间片进行动态调整。

CFS 使用红黑树作为运行队列

公平共享调度在每次调度时，需要从运行队列中选取当前虚拟时间最短的任务。因此，相比于将以虚拟时间排序的队列作为运行队列，CFS 使用了红黑树。虽然 CFS 能以 $O(1)$ 复杂度从队列和红黑树中取出当前虚拟时间最短的任务，但是在维护运行队列方面，红黑树的时间开销更少。这是由于红黑树是一种有序平衡的二叉查找树，根据索引键能以 $O(\log N)$ 的时间复杂度在红黑树中插入一个节点 (其中 N 为红黑树的键值数量)，而向队列数据结构插入一个元素的时间复杂度为 $O(N)$ 。

如图 7.18 所示，cfs_rq 记录了红黑树的根节点 (tasks_timeline^②)，可以根据红黑树节点 (run_node) 找到对应任务的数据结构 (task_struct) 进行调度。另外，为了能够以 $O(1)$ 复杂度找到当前虚拟时间最短的任务，CFS 维护了指向红黑树中虚拟时间最短任务的指针 (rb_leftmost)。CFS 的红黑树仅维护当前正在运行和可运行的

① 以 Linux 5.7.8 版本为例，文件 kernel/sched/code.c 中定义了 sched_prio_to_weight 数组，表示静态优先级与任务权重的对应关系。

② CFS 中红黑树根节点的变量名 tasks_timeline 实际上对应虚拟时间轴，而 CFS 的每次调度决策就是在选取时间轴上虚拟时间最短的任务。

任务，不会记录其他状态的任务，从而减少了插入红黑树的开销。

CFS 阻塞任务唤醒

当一个任务在一段时间内由于阻塞或睡眠而没有运行时，其虚拟时间是不会增加的。当它再一次进入可运行状态时，很有可能会在很长一段时间内占据 CPU 核心。与步幅调度（7.5.2 节）相同，调度器会在一个任务被重新插入运行队列时，将该任务的虚拟时间设置为该任务当前虚拟时间与运行队列中任务的最小虚拟时间中的较大值。这样，既保证了阻塞后的任务会尽量被优先调度，又避免了该任务长时间占用 CPU 时间。

在 CFS 进入 Linux 主线后，随着 Linux 版本从 2.x 演进到 5.x，CFS 被证明经受住了时间的考验。在 Linux 的版本更迭中，CFS 也在不断改进和完善，这也从侧面反映了设计和实现一个综合、高效的调度器并非易事。

7.7.4 Linux 的细粒度负载追踪

上一节描述的 CFS 相关机制，可以对应到 Linux 中每个 CPU 核心的本地队列。而在负载均衡方面，Linux 面临的一大挑战是如何确定当前任务的负载情况。对于当前任务的负载信息描述得越准确，对应的负载均衡策略就越有效。在大部分场景中，多个任务所需的计算量是不同的，因此 7.6.2 节描述的基于运行队列长度的负载均衡机制无法应用于 Linux。同时，一个任务的执行负载是动态变化的，因此系统必须动态追踪当前负载，但这会造成一定的性能开销。所以，如何在保持低开销的同时对负载进行精确追踪是调度器设计和实现的一大挑战。本节将介绍 Linux 中目前使用的调度实体粒度负载追踪（Per-Entity Load Tracking, PELT）机制^[5]。

第一种是运行队列粒度的负载追踪。在 Linux 3.8 版本以前，内核以每个 CPU 核心的运行队列为粒度来计算负载，认为运行队列长则负载高，导致负载追踪不够精确。当调度器决定从一个高负载的运行队列中选取一个任务迁移到低负载的运行队列时，由于缺乏每个任务的信息，因此无法选出合适的任务进行迁移。所以，以运行队列为粒度的负载追踪并不能适应负载均衡的要求。

第二种是调度实体粒度的负载追踪。在 3.8 版本以后，Linux 使用了以描述调度实体（单个任务）为粒度的负载计算方式，做到了更细粒度的负载追踪。PELT 通过记录每个任务的历史执行状况来表示任务的当前负载。具体地，调度器会以 1 024 微秒为一个周期，记录任务处于可运行状态（包括正在运行以及等待被运行）的时间，记为 x 微秒。在该任务的第 i 个周期（ T_i ）内，任务对当前 CPU 的利用率为 $x/1\,024$ ，而对应的负载 L_i 为 $\text{scale_cpu_capacity} \times x/1\,024$ ，其中 $\text{scale_cpu_capacity}$ 是 CPU 容量（用于归一化系统中 CPU 的处理能力），可以理解为对应 CPU 核心的处理能力。

在收集到任务每个周期内的负载后，PELT 需要计算任务一段时间内所有周期的累计负载，记为 L 。一种计算 L 的简单方法是将所有周期的负载直接累加。然而，一个任务

可能运行一天甚至更久,对于计算任务的当前负载的 PELT 来说,过于久远的历史负载信息没有太大参考意义,因此应该记录一定周期范围内的累计负载。另外,距离当前时间点越近的周期所记录的负载更有可能反映任务的当前执行负载,所以应该对累计负载有更大的贡献。因此,PELT 采用的计算任务累计负载的方式是通过衰减系数 y 来衰减之前周期的负载,具体公式如下:

$$L = L_n + L_{n-1} y + L_{n-2} y^2 + L_{n-3} y^3 + \dots$$

其中, L_i 代表第 i 个周期 (T_i) 的负载, L_n 为当前周期 T_n 的负载。该计算公式不仅考虑了不同周期的权重,而且开销也很低。这是因为 PELT 无须保存任务每个周期内的负载,只需要维护累计负载 L 。当需要计算新的累计负载时,PELT 只需要用衰减系数 y 衰减原累计负载 L 并与 L_n 相加即可:

$$L' = L y + L_n$$

通过计算每个任务的负载 L ,PELT 就可以统计出每个运行队列的负载,以便调度器做出有效的迁移决策。

总体来说,PELT 的好处是开销很小,且能帮助调度器更细粒度、更精确地监测任务的负载,对于预测负载均衡策略的效果有很大帮助。

7.7.5 Linux 的 NUMA 感知调度

5.3.5 节介绍了服务器中常见的 NUMA 架构,Linux 内核中的调度策略也考虑了 NUMA 环境^[6]。对于多核处理器硬件,Linux 内核中的调度器主要需要权衡两个问题:一方面,需要尽可能让任务均匀地分布在系统的各个核心上,达到更好的负载均衡;另一方面,频繁地在不同的核心之间迁移任务意味着丧失了任务的本地性,特别是在 NUMA 环境下,跨 NUMA 节点的任务迁移将导致巨大的迁移开销。因此 Linux 内核引入了调度域 (Scheduling Domain) 的概念,将 CPU 分成了不同的调度域。每个调度域是具有相同属性的一组 CPU 的集合,并根据硬件特征划分成不同的层级,呈现出一种树状结构。

图 7.19 展示了一个 NUMA 系统的调度域。最下层是逻辑核调度域,每个域包含同一个物理核中的逻辑核,这些逻辑核共享高速缓存,因此在它们之间迁移的开销最低。向上一层是核调度域,每个域包含共享同一个 L2 高速缓存的所有核心。再向上分别是处理器调度域(包含同一个处理器中的所有核心)、NUMA 节点调度域(包含一个 NUMA 节点中的所有核心)和全系统调度域(包含整个系统中的所有核心)。内核在启动时会把 CPU 根据其拓扑结构归为不同的域。不同的域有不同的负载均衡周期,越下层的域,任务在核心之间迁移的开销越低,执行负载均衡越频繁。到 NUMA 节点调度域这一层,负载均衡就很少执行。通过划分调度域的方法,就可以在实现一定负载均衡的同时,避免在 NUMA 节点之间频繁迁移任务。

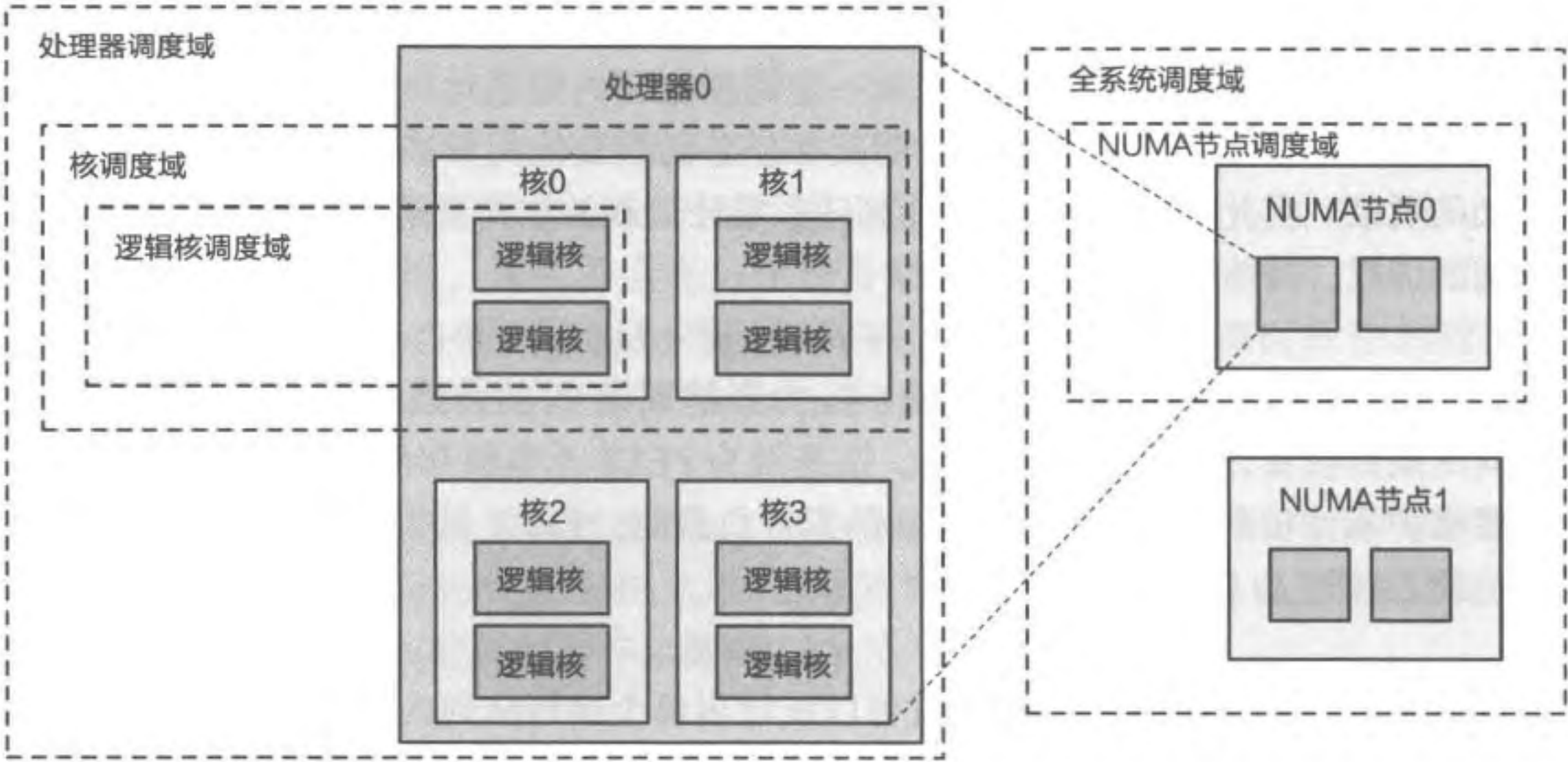


图 7.19 NUMA 系统的调度域

7.8 思考题

1. 小明希望利用暑假时间在 ChCore 上运行许多数据分析与深度学习程序，并希望在开学后一并查看计算结果。
- (a) 小明在思考应该在 ChCore 中实现抢占式调度还是非抢占式调度，请帮小明进行选择并给出原因。
 - (b) 在选取调度策略时，小明选择了“先到先得”策略。试分析小明的这一选择可能带来的好处。然而，这种策略可能导致 I/O 任务、CPU 任务混合场景出现问题，请简要描述可能的原因，并帮助小明提出可能的解决方案。
 - (c) 小明希望允许其他同学一同使用他的计算资源进行数据分析工作。为此，小明为 ChCore 添加了对权重的支持，在 ChCore 上实现了步幅调度。假设整个系统仅需要执行如下三个拥有不同 Ticket 的任务（表 7.7），请给出一个合适的 MaxStride 值并填写表 7.8，给出后续调度周期中被调度的任务。

表 7.7 三个任务的 Ticket 值

任务	Ticket
A	30
B	40
C	60

表 7.8 任务调度序列

次序	1	2	3	4	5	6	7	8	9
任务	A	B	C	?	?	?	?	?	?